

Project TeamworkTemplate

Version 1 9/11/24

A **separate copy** of this template should be filled out and submitted by each student, regardless of the number of students on the team. Also change the title of this template to “Project x Teamwork <team> - <netid>”

1	Team Name: VFC													
2	Individual name: Victoria Caruso													
3	Individual netid: vcaruso													
4	Other team members names and netids:													
5	Link to github repository: https://github.com/victoriafcaruso/Project1-TOC													
6	Overall project attempted, with sub-projects: NTM Trace													
7	List of included files (if you have many files of a certain type, such as test files of different sizes, list just the folder): (Add more rows as necessary) <table border="1"> <thead> <tr> <th>File/folder Name</th> <th>File Contents and Use</th> </tr> </thead> <tbody> <tr> <td colspan="2">Code Files</td> </tr> <tr> <td>ntm_tracer.py</td> <td>This file contains the run and and print_trace_path definitions. These functions are called on entryptoint.py. And the run definition calls on the print_trace_path to display the path taken by the NTM when the string is accepted. The run definition follows the instructions listed in the skeleton code.</td> </tr> <tr> <td colspan="2">Test Files</td> </tr> <tr> <td>tester_1.csv</td> <td>This file contains the information for the nondeterministic turing machine that only accepts strings with two 1's in it that are not consecutive. It follows the csv format specified in the Project 2 instructions under “Header Lines.” Rejected Strings: 001, 0110, 0000 Accepted Strings: 111, 101, 000101</td> </tr> <tr> <td>tester_2.csv</td> <td>This file contains the information for the deterministic turing machine that only accepts strings with an equal number of 0's and 1's. It follows the csv format specified in the Project 2 instructions under “Header Lines.” The input string must start with “2”</td> </tr> </tbody> </table>		File/folder Name	File Contents and Use	Code Files		ntm_tracer.py	This file contains the run and and print_trace_path definitions. These functions are called on entryptoint.py. And the run definition calls on the print_trace_path to display the path taken by the NTM when the string is accepted. The run definition follows the instructions listed in the skeleton code.	Test Files		tester_1.csv	This file contains the information for the nondeterministic turing machine that only accepts strings with two 1's in it that are not consecutive. It follows the csv format specified in the Project 2 instructions under “Header Lines.” Rejected Strings: 001, 0110, 0000 Accepted Strings: 111, 101, 000101	tester_2.csv	This file contains the information for the deterministic turing machine that only accepts strings with an equal number of 0's and 1's. It follows the csv format specified in the Project 2 instructions under “Header Lines.” The input string must start with “2”
File/folder Name	File Contents and Use													
Code Files														
ntm_tracer.py	This file contains the run and and print_trace_path definitions. These functions are called on entryptoint.py. And the run definition calls on the print_trace_path to display the path taken by the NTM when the string is accepted. The run definition follows the instructions listed in the skeleton code.													
Test Files														
tester_1.csv	This file contains the information for the nondeterministic turing machine that only accepts strings with two 1's in it that are not consecutive. It follows the csv format specified in the Project 2 instructions under “Header Lines.” Rejected Strings: 001, 0110, 0000 Accepted Strings: 111, 101, 000101													
tester_2.csv	This file contains the information for the deterministic turing machine that only accepts strings with an equal number of 0's and 1's. It follows the csv format specified in the Project 2 instructions under “Header Lines.” The input string must start with “2”													

		Rejected Strings: 200, 211, 2001 Accepted Strings: 20101, 210, 21100
	composite.csv	This test file contains the TM for the Composite example in the TM sim file.
	Output Files	
		N/A 4.7 Output All runs should first echo the name of the machine (from line 1), the initial string, the depth of the tree of configurations, and the total number of transitions simulated. If the machine is deterministic the latter two numbers should be the same. If the simulation halts because of reaching an accept configuration, print out the following: • "String accepted in " and then the number of transitions from the start to the accept on just the accepting path (the depth of the tree). • Starting at the starting configuration, print out each configuration for each level in the format: left of head string, state, head character and right of string. For example one line of output might be "Level 3", and there is a following line for each configuration at that level. If the simulation halts because of all paths lead to reject, "String rejected in ", followed the number of steps from the start to the last reject (this should be the max depth). If the step limit is exceeded, print out something like "Execution stopped after" the max step limit. For example inputting the string "aaaaa" for the <i>aplus</i> machine should accept at a depth of about 6 and the number of transitions should be somewhere between 6 and 15. All output is printed in the terminal, rather than into another file
	Plots (as needed)	
		N/A
8	Individual Student time (in hours) to complete: 15	
9	Your specific activities and responsibilities: Since I worked alone on this project, I was responsible for all aspects of its completion. This includes understanding the architecture of the skeleton code and how existing files called on one another, writing the print_trace_path and completing run definitions in ntm_trace.py, and creating the tester_1.csv and tester_2.csv files. I dedicated a significant amount of time to design and debug the definitions listed above.	
10	What was personally learned (topic, programming, algorithms) I learned more about how NTMs function and go through possible "paths" step by step to find the first path that leads to the accepted state. I also learned more about how to code in python, and specifically work with data structures like lists, trees, and dictionaries; while I have leveraged these structures before, this provided great experience to gain a better understanding of how to leverage such data structures. I also gained a better understanding of how to trace back paths taken in trees/maps, such as that modeled by the transitions for the ntms.	
11	How team was organized, and what might be improved. Since I worked on this project alone, I was responsible for all of the code, testing, and	

	working with github; nothing to be improved.
12	Any additional material: N/A